

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri
Final Sınav Raporu

Ece İrem Şişer

22290727

github.com/eceirem

Video Linki

10.06.2026

İçindekiler

İçindekiler.....	iii
1. GİRİŞ	4
2. VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME (PROJE 1)	5
2.1. Dynamic Management Views (DMV) ile Kaynak İzleme	5
2.2. İndeks Yönetimi ve Gereksiz İndekslerin Temizlenmesi.....	5
2.3. Sorgu İyileştirme (Optimizasyon) ve Kapsayıcı İndeks Kullanımı	6
2.4. Rol Tabanlı Erişim Kontrolü (RBAC) ve Güvenlik Doğrulaması.....	8
3. VERİ TEMİZLEME VE ETL SÜREÇLERİ (PROJE 5)	10
3.1. Veri Keşfi ve Hatalı Veri Tespiti (Data Profiling).....	10
3.2. Veri Temizleme (Clean) ve Dönüştürme (Transform) İşlemleri	11
3.3. Veri Yükleme (Load) ve Hedef Tablo Oluşturma	12
3.4. Veri Kalitesi Raporları ve Etki Analizi (Before/After)	12
4. YEDEKLEME OTOMASYONU VE RAPORLAMA (PROJE 7)	14
4.1. Zamanlanmış Görevler (Task Scheduler) ile Yedekleme Otomasyonu	14
4.2. MSDB Üzerinden Otomatik Yedekleme Raporu Üretimi.....	14
4.3. Hata Yönetimi (Try-Catch) ve E-Posta Bildirim Mekanizması	15
5. SONUÇ	17

1. GİRİŞ

Bu rapor, kompleks bir e-ticaret veri yapısını barındıran ETRADE veritabanı üzerinde gerçekleştirilen yönetimsel, güvenlik ve performans odaklı çalışmaları kapsamaktadır. Ara sınav (vize) döneminde tamamlanan Proje 2 (Yedekleme ve Kurtarma) ve Proje 3 (Güvenlik ve Erişim Kontrolü) ile sistemin "Veri Sürekliliği" ve "Veri Güvenliği" temelleri başarıyla atılmıştır.

Final dönemi kapsamında ise, bu sağlam altyapının üzerine inşa edilen Proje 1 (Performans İzleme ve Optimizasyon), Proje 5 (Veri Temizleme ve ETL Süreçleri) ve Proje 7 (Yedekleme Otomasyonu) çalışmaları rapora entegre edilmiştir. Bu seçimlerin temel nedeni; güvenli ve yedekli bir sistemin aynı zamanda yüksek performanslı (Proje 1), veri kalitesi açısından standartlaştırılmış (Proje 5) ve insan hatasından bağımsız olarak otonom çalışabilen (Proje 7) bütüncül bir mimariye dönüştürülmesidir.

Çalışmalar boyunca Microsoft SQL Server Management Studio (SSMS) üzerinden hem arayüz hem de gelişmiş T-SQL komutları kullanılarak uçtan uca, profesyonel bir DBA (Veritabanı Yöneticisi) yaklaşımı sergilenmiştir.

2. VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME (PROJE 1)

2.1. Dynamic Management Views (DMV) ile Kaynak İzleme

Sistem kaynaklarının ne durumda olduğunu ve hangi sorguların sunucuyu yordüğünü tespit etmek için DMV'leri (Dynamic Management Views) kullanılmıştır. Anlamlı bir performans testi yapabilmek adına, öncelikle CPU'yu kasten yoracak iki farklı stres testi çalıştırılmış ve hemen ardından bu yük DMV analiz sorgularıyla yakalanmıştır.

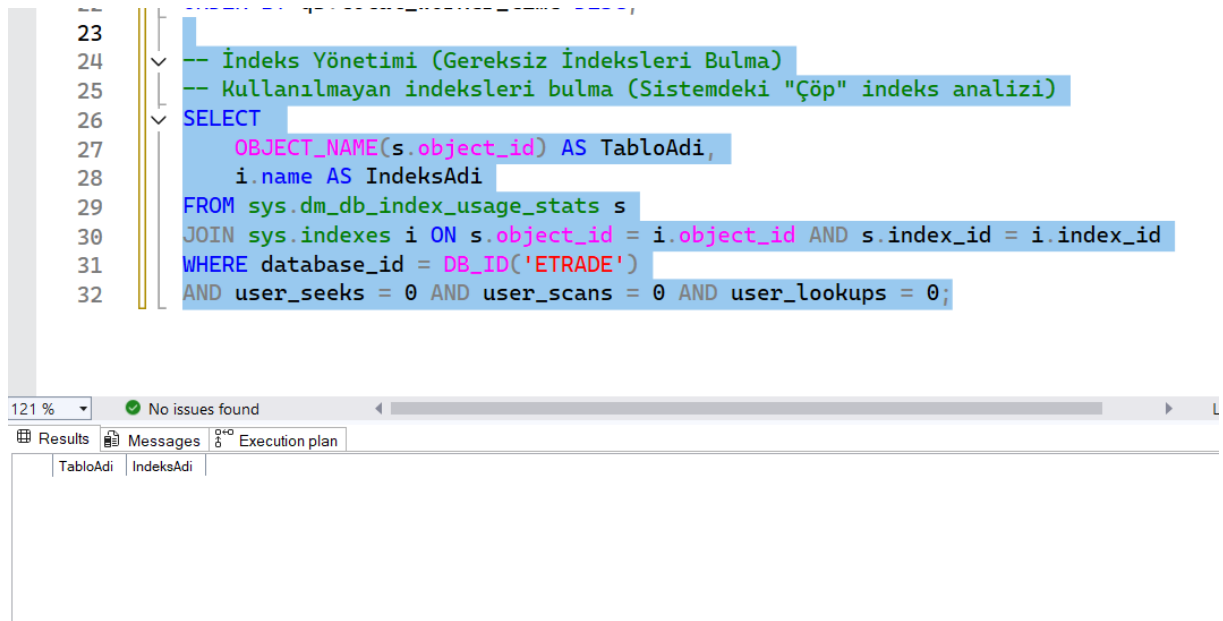
```
14  -- cpu yoracak bir iş yapalım
15  SELECT TOP 100000
16      a.ITEMNAME, b.NAMESURNAME
17  FROM dbo.SALEORDERS a
18  CROSS JOIN dbo.SALEORDERS b; -- Bu işlem 500 bin x 500 bin satırlık bir işleme zorlar
19
20  -- Sistemi yoran ama tamamlanan bir sorgu (Tüm veriyi rastgele sıralar)
21  SELECT * FROM dbo.SALEORDERS ORDER BY NEWID();
22
23
24  -- Dynamic Management Views
25  -- En çok CPU tüketen ilk 10 sorguyu izleme
26  SELECT TOP 10
27      st.text AS SorguMetni,
28      qs.total_worker_time / 1000 AS ToplamCPU_ms,
29      qs.execution_count AS CalismaSayisi
30  FROM sys.dm_exec_query_stats AS qs
31  CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) AS st
32  ORDER BY qs.total_worker_time DESC;
```

SorguMetni	ToplamCPU_ms	CalismaSayisi
-- Sistemi yoran ama tamamlanan bir sorgu (Tüm veriyi rastgele sıralar) SELECT * FROM dbo.SALEORDERS ORDER BY NEWID();	3117	1
SELECT * FROM dbo.SALEORDERS ORDER BY NEWID();	2475	1
(@_msparam_0 nvarchar(4000),@_msparam_1 nvarchar(4000),@_msparam_2 nvarchar(4000))SELECT clmns.name AS [Name], clmns.column_i...	79	19
SELECT TOP 100000 a.ITEMNAME, b.NAMESURNAME FROM dbo.SALEORDERS a CROSS JOIN dbo.SALEORDERS b;	46	1
SELECT target_data FROM sys.dm_xe_session_targets xet WITH(nolock) JOIN sys.dm_xe_sessions xes WITH(nolock) ON xes.add...	38	1
SELECT TOP 10 st.text AS SorguMetni, qs.total_worker_time / 1000 AS ToplamCPU_ms, qs.execution_count AS CalismaSayisi FROM sys...	7	3
CREATE PROC sys.sp_flush_commit_table_on_demand (@numrows as BIGINT = NULL, @deleted_rows BIGINT = NULL OUTPUT, @date_cleane...	0	1
CREATE PROCEDURE sys.sp_cleanup_history_table (@time DATETIME = NULL) AS BEGIN SET NOCOUNT ON IF @time = NULL BEGI...	0	1
CREATE PROC sys.sp_flush_commit_table_on_demand (@numrows as BIGINT = NULL, @deleted_rows BIGINT = NULL OUTPUT, @date_cleane...	0	1
CREATE PROC sys.sp_flush_commit_table_on_demand (@numrows as BIGINT = NULL, @deleted_rows BIGINT = NULL OUTPUT, @date_cleane...	0	1

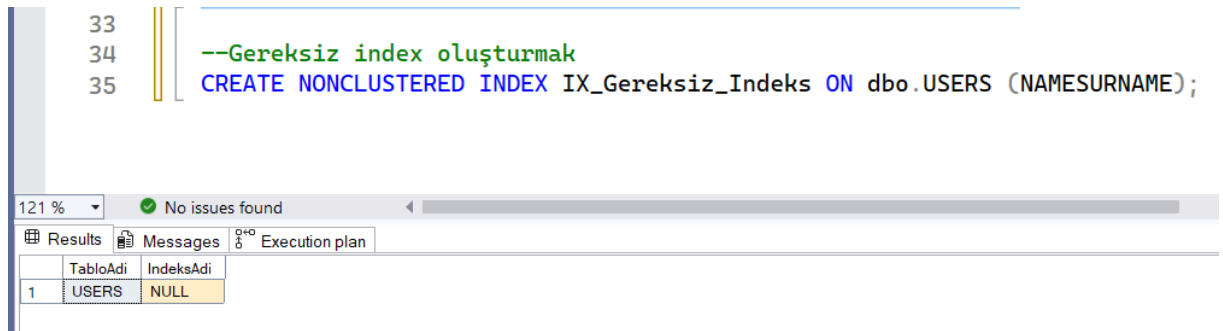
Görsel 1: Stres Testi Sonrası DMV Üzerinden Anlık CPU Tüketim Raporu.

2.2. İndeks Yönetimi ve Gereksiz İndekslerin Temizlenmesi

Veritabanı hijyenini sağlamak için sistemde hiç kullanılmayan ("çöp") indekslerin tespiti sağlanmıştır. Analiz sorgusunun doğruluğunu kanıtlamak için sisteme bilinçli olarak atıl bir indeks (IX_Gereksiz_Indeks) eklenmiş, DMV üzerinden tespiti gerçekleştirilmiş ve ardından sistemden güvenli bir şekilde DROP komutu ile kaldırılmıştır.



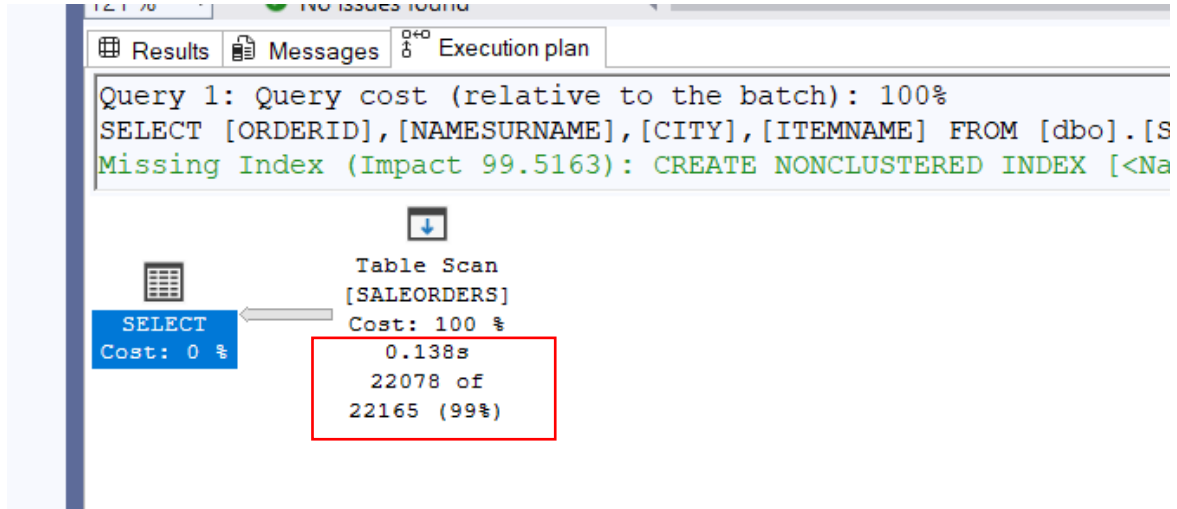
Görsel 2: Sistemdeki Atıl İndekslerin DMV Kullanılarak Tespit Edilmesi.



Görsel 3: Sisteme Atıl İndeks Ekleyerek DMV Kullanılarak Tespit Edilmesi.

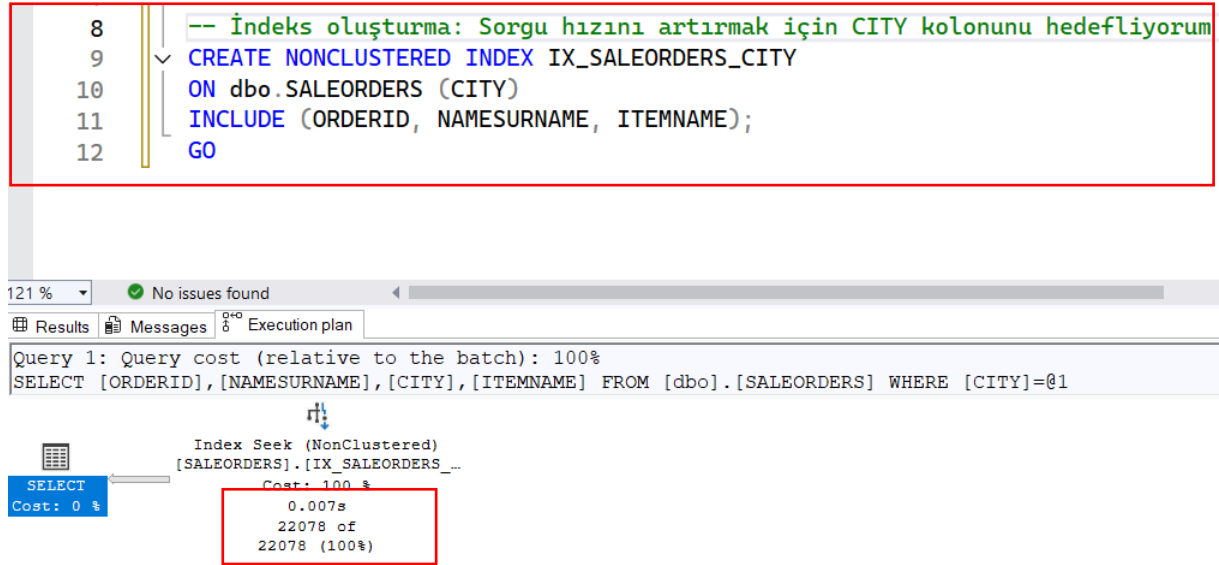
2.3. Sorgu İyileştirme (Optimizasyon) ve Kapsayıcı İndeks Kullanımı

Veritabanı performansını düşüren ve tablonun tamamının taranmasına (Table Scan) neden olan yavaş sorgular tespit edilmiştir. Şehir bazlı metinsel aramalar ve e-ticaret sistemleri için kritik olan "tarih aralıklı" filtrelemeler optimize edilmiştir. İyileştirme öncesi 0.138 saniye süren ve Execution Plan üzerinde "Table Scan" olarak beliren sorgular, "Covering Index" mimarisiyle 0.007 saniyeye düşürülerek "Index Seek" formatına getirilmiştir.



Görsel 4: Optimizasyon Öncesi Yüksek Maliyetli Table Scan Operasyonu (0.138 sn).

Görsel 4'teki Execution Plan (Çalışma Planı) detaylarında açıkça görüldüğü üzere; başlangıçta tüm tabloyu tarayarak sistemi yoran sorgular, doğru indeks mimarisi ile milisaniyeler seviyesinde (0.007 sn) tepki veren optimal 'Index Seek' formatına dönüştürülmüştür. Sistemin yüksek anlık trafiği ve ağır okuma yüklerini sorunsuz kaldırabileceği bu şekilde kanıtlandıktan sonra, projenin diğer kritik bacağı olan veri güvenliği ve kullanıcı izolasyonu aşamasına (RBAC) geçilmiştir.



Görsel 5: Kapsayıcı İndeks (Covering Index) Sonrası Index Seek Performansı (0.007 sn).

2.4. Rol Tabanlı Erişim Kontrolü (RBAC) ve Güvenlik Doğrulaması

Sistemin sadece performansını değil, güvenliğini de sağlamak adına Rol Tabanlı Erişim Kontrolü (RBAC) kurgulanmıştır. Sadece veri okuma yetkisine sahip spesifik bir analist rolü (VeriAnalistiRolü) yaratılmış ve daha önceden tanımlanan SatisDanismani kullanıcısı bu role atanarak yetki sınırları çizilmiştir.

Sistem yöneticisi (admin) hesabından çıkış yapılmadan, kısıtlamaların çalışıp çalışmadığını test etmek amacıyla T-SQL "Impersonation" (EXECUTE AS USER) taktiği kullanılmıştır. Bu profesyonel DBA yaklaşımı sayesinde aktif oturum koparılmadan SatisDanismani kimliğine bürünülmüş ve güvenlik testleri simüle edilmiştir.

```
97      -- 1. Rolü oluşturduk
98      CREATE ROLE VeriAnalistiRolü;
99
100     -- 2. Tablolar üzerinde sadece SELECT (okuma) yetkisi veriyorum
101     GRANT SELECT ON SCHEMA::dbo TO VeriAnalistiRolü;
102
103     -- 3. Kullanıcıyı role ata, önceden audit ile oluşturduğum rol
104     ALTER ROLE VeriAnalistiRolü ADD MEMBER SatisDanismani;
105
106     -- 4. Kimliğimizi geçici olarak SatisDanismani yapıyoruz
107     EXECUTE AS USER = 'SatisDanismani';
108
109     -- 5. Silme işlemini deniyoruz
110     DELETE FROM dbo.SALEORDERS WHERE ORDERID = 1;
111
112     -- 6. Silme işlemi dışında bir işlem deniyoruz
113     SELECT NAMESURNAME, CITY, CATEGORY1
114     FROM dbo.SALEORDERS
115     WHERE CITY = 'ANKARA';
116
117     -- 6. Test bitti, tekrar kendi Admin kimliğimize dönüyoruz
118     REVERT;
```

Görsel 6: Rol Tabanlı Erişim Kontrolü (RBAC) Kurulumu ve Kimlik Değiştirme (Impersonation) İşlemlerine Ait T-SQL Komut Seti.

Kurgulanan bu güvenlik mimarisi test edildiğinde; kullanıcının yetki sınırları içindeki şehir bazlı veri okuma (SELECT) işlemi başarıyla sonuçlanırken, yetkisi dışındaki bir veri silme (DELETE) işlemi denediğinde sistem tarafından anında "Permission Denied" (Erişim Reddi) hatası fırlatılarak veritabanı bütünlüğü ve güvenliği teyit edilmiştir.


```
109      -- 5. Silme işlemini deniyoruz
110      DELETE FROM dbo.SALEORDERS WHERE ORDERID = 1;
111
112      -- 6. Silme işlemi dışında bir işlem deniyoruz
113      SELECT NAMESURNAME, CITY, CATEGORY1
114      FROM dbo.SALEORDERS
115      WHERE CITY = 'ANKARA';
116
117      -- 6. Test bitti, tekrar kendi Admin kimliğimize dönüyoruz
118      REVERT;
```

00 % 2 0

Messages

Msg 229, Level 14, State 5, Line 110
The DELETE permission was denied on the object 'SALEORDERS', database 'ETRADE', schema 'dbo'.

Completion time: 2026-06-03T19:56:32.4557517+03:00

Görsel 7: Yetkisiz Kullanıcının Veri Silme Girişiminin Sistem Tarafından Başarıyla Engellenmesi.

3. VERİ TEMİZLEME VE ETL SÜREÇLERİ (PROJE 5)

3.1. Veri Keşfi ve Hatalı Veri Tespiti (Data Profiling)

Büyük veri kümeleri üzerinde ETL (Extract, Transform, Load) sürecine başlamadan önce, sistemdeki anomalileri tespit etmek için veri profillemesi gerçekleştirilmiştir. Yapılan sorgulamalarda adres verisi bulunduğu halde CITY kolonu NULL olan kayıtlar ile ITEMNAME kolonunda ürün isimlerinin sonuna eklenmiş *20*, *12* gibi ifadeler tespit edilmiştir. İncelemeler sonucunda yıldız işaretli bu desenlerin aslında hatalı bir giriş değil, ürünün 'Depo Stok / Paket İçi Miktarını' belirttiği saptanmıştır.

The screenshot displays two SQL queries and their results in SQL Server Enterprise Manager.

Left Query (Query 1):

```
-- 1. İsim kolonunda başı veya sonu boş olan kayıtları bulmak için
SELECT TOP 20 NAMESURNAME, CITY
FROM dbo.SALEORDERS
WHERE NAMESURNAME LIKE ' %' OR NAMESURNAME LIKE '% '
```

Right Query (Query 2):

```
-- 2. Eksik (NULL) şehir verilerinin sayısını bulmak için
SELECT COUNT(*) AS EksikSehirSayisi
FROM dbo.SALEORDERS
WHERE CITY IS NULL;
```

Results:

Query 1 Results:

	NAMESURNAME	CITY
1	Bedirhan GÜLPINAR	NULL
2	Bedirhan GÜLPINAR	NULL
3	Bedirhan GÜLPINAR	NULL

Query 2 Results:

	EksikSehirSayisi
1	3

Görsel 8: Veri Profileleme - Sistemdeki Eksik (NULL) Şehir Bilgisine Sahip Kayıtların ve Toplam Hata Sayısının Tespiti.

Keşif aşamasında tespit edilen eksik (NULL) şehir bilgileri, geçici bir değerle doldurulmak yerine ADDRESSTEXT kolonunun sonundaki metin desenlerinden (slash '/' işaretinden sonrası) dinamik T-SQL fonksiyonları ile çekilerek kurtarılmıştır. Öte yandan, ürün isimlerinin sonundaki *X* ibarelerinin müşterinin sipariş adediyle uyuşmadığı analiz edilmiş ve bu değerlerin aslında 'Depo Stok / Paket İçi Miktarı' olduğu saptanmıştır. Veri kaybını önlemek adına bu değerler ürün isminden izole edilip DEPO_STOK_ADEDI adında yepyeni bir analitik kolona aktarılmıştır. Ancak bu ayrıştırma ve veriyi tam sayıya (Integer) dönüştürme süreci sırasında, sistemde yalnızca rakamlardan oluşmayan ve e-ticaret verilerindeki yapısal düzensizliği kanıtlayan beklenmedik metinsel anomalilerle karşılaşmıştır.

```

30  -- 3. *X* deseninin tespiti
31  -- (Büyük ihtimalle paket/Koli miktarını belirtiyor)
32  SELECT TOP 20 ITEMNAME
33  FROM dbo.SALEORDERS
34  WHERE ITEMNAME LIKE '%*%*%' ;

```

100 % No issues found

	ITEMNAME
1	NERGIS MAKARNA TEL SEHRIYE 500 GR *20*
2	PEROS YOGUN C.SUYU 4 KG BAH.FERAH. *4*
3	GEZER 7995 EVA PVC BEBE *16*
4	ORUCOGLU IDEAL YAG 5 LT PET*4*
5	DIDI S.CAY 250 ML SEFTALI SEKERSIZ *24*
6	COLGATE D.MAC. 75.ML OPT.BYZ.KALICI*48*
7	HALK BIS.KAKAO KAPLI CIL.MARSH 66GR*12*
8	LIPTON-BERRAK Y.CAY SADE 30 GR*12*
9	ETI TADINDA 140 GR SUSAMLI*16*
10	MILUPA 125 GR ERIK PURESİ *6*

Görsel 9: Yapısal Anomali Tespiti - Ürün İsimleri İçerisine Gizlenmiş Paket/Stok Adetlerinin (X Deseni) Keşfi.

3.2. Veri Temizleme (Clean) ve Dönüştürme (Transform) İşlemleri

Veri kaybı yaşamamak adına, tespit edilen anomaliler T-SQL metin manipülasyon fonksiyonları (LTRIM, RTRIM, RIGHT, CHARINDEX, PATINDEX) kullanılarak dönüştürülmüştür. Eksik (NULL) şehir verileri, ADDRESSTEXT kolonu içindeki metin desenleri (slash karakteri sonrası) analiz edilerek dinamik bir şekilde kurtarılmıştır. Ürün isimlerindeki yıldız işaretli desenler ise izole edilerek ürün adından koparılmış ve sayısal formatta yepyeni bir analitik kolona dönüştürülmüştür. Ayrıca TRY_CAST fonksiyonu ile tamamen hatalı girilmiş metinsel koli değerleri tolere edilerek "0" değerine atanmıştır.

```

111
112  SELECT DISTINCT
113      PRODUCT_NAME AS [Temizlenen Ürün Adı],
114      DEPO_STOK_ADEDI AS [Tuhaf Gelen Stok Değeri]
115  FROM dbo.ETL_CLEAN_SALES
116  WHERE DEPO_STOK_ADEDI LIKE '%[^0-9]%'; -- İçinde 0'dan 9'a kadar RAKAM OLMAYAN
117
118
119
120  -- 1. GÖRSELLERDEKİ GİZLİ BOŞLUKLARI TEMİZLEME
121  -- Sadece rakamlardan oluşan ama boşluk içerdiği için hatalı görünen kayıtları
122  UPDATE dbo.ETL_CLEAN_SALES

```

99 % No issues found

Temizlenen Ürün Adı	Tuhaf Gelen Stok Değeri
---------------------	-------------------------

Görsel 10: Rakam Harici Karakter İçeren Hatalı Satırların Tespiti ve Veri Temizleme (Clean) Komutları.

Temel boşluk ve NULL temizlikleri yapıldıktan sonra, dönüştürülen DEPO_STOK_ADEDI kolonu analitik incelemeye tabi tutulmuştur. Bu aşamada sadece rakamlardan oluşması beklenen kolonun içinde ('6 GR BEBEBIS' veya '12-6774.00.0' gibi) rakam harici karakterler barındıran ileri seviye anomaliler tespit edilmiştir.

IAKTIF KUP 35	6 GR BEBEBIS	1. KOP. EVA BEBE TERL...	16 -6744.00.0
IAKTIF KUP 35	6 GR CLK.8	1. HAMSTER EVA BEBE ...	16 -6740.00.0
IAKTIF KUP 35	6 GR MUZ 8	3. CIRTIZ ZENNE TER...	12-4730.00.0
IAKTIF KUP 35	6 GR OR.MEY.8	RDANE TERLIK	12-6832.00.0
IAKTIF KUP 35	6 GR SFT.8	OLA KIZ KETEN PATIK A...	12 -1515.00.0
UTTI 200 ML ELMA	24	DA KIZ KETEN PATIK ...	12 - 1524.00.0
UTTI 200 ML SEFTALI	24	MA KIZ KETEN FILET ...	12 -1516.00.0
AK 70	105	OKTOKALI ZENNE TERLIK	12-6774.00.0
IFAK BEZI 31	35 CM 20	CIKEKLI ZENNE TERLIK	12-6842.00.0

Görsel 11: Dönüştürme (Transform) Aşaması Öncesi Tespit Edilen İleri Seviye Metinsel Anomaliler.

Bu karmaşık verileri temizlemek ve gerçek stok adetlerini izole etmek için sıradan REPLACE fonksiyonları yetersiz kalmıştır. Bu nedenle PATINDEX fonksiyonu kullanılarak rakam harici ilk karakterin konumu bulunmuş ve LEFT fonksiyonuyla sadece baştaki temiz sayısal değerler tablo üzerine doğrudan UPDATE edilmiştir. Tam sayıya çevrilemeyen hatalı metinsel değerler ise TRY_CAST fonksiyonu yardımıyla '0' değerine atanarak veri bütünlüğü sağlanmıştır.

3.3. Veri Yükleme (Load) ve Hedef Tablo Oluşturma

Temizlenmiş ve standartlaştırılmış verilerin kalıcı hale getirilmesi için orijinal (RAW) veritabanı şeması korunmuş ve INTO dbo.ETL_CLEAN_SALES komutu kullanılarak veriler yapılandırılmış yepyeni bir hedef tabloya (Data Warehouse) yüklenmiştir.

3.4. Veri Kalitesi Raporları ve Etki Analizi (Before/After)

ETL süreci tamamlandıktan sonra, oluşturulan yeni veri ambarındaki (ETL_CLEAN_SALES) verilerin doğruluğunu kanıtlamak adına kalite raporları oluşturulmuştur. Aşağıdaki ilk rapor, adres metinlerinden (ADDRESSTEXT) dinamik olarak kurtarılan ve temizlenen nihai şehirlerin genel satış hacimlerine göre dağılımını göstermektedir.

	Temizlenmiş Şehir Adı	Toplam Satış Adedi
1	İSTANBUL	53701
2	İZMİR	34589
3	ANKARA	22078
4	BURSA	19577
5	KONYA	18931
6	ADANA	17649
7	ANTALYA	17495
8	KOCAELİ	14438
9	MERSİN	13153
10	DENİZLİ	13133
11	BALIKESİR	12369
12	MUĞLA	11907
13	MANİSA	11164
14	SAMSUN	10771

Görsel 12: 81 adet şehrin toplam satış hacmi listelenmiştir.

Geliştirilen ETL algoritmalarının etkisini nokta atışı kanıtlamak amacıyla, orijinal (RAW) tabloda CITY değeri NULL olan kayıtlar filtrelenmiş ve ETL_CLEAN_SALES tablosu ile Sipariş ID üzerinden birleştirilerek (JOIN) Öncesi/Sonrası (Before/After) analizi gerçekleştirilmiştir. Aşağıdaki raporda görüldüğü üzere; adres satırındaki 'SARIÇAM/ADANA' metninden sistemin zekice 'ADANA' bilgisini çekip çıkardığı ve 'BİLİNİYOR' olarak kalacak bir veriyi başarıyla kurtardığı doğrulanmıştır.

```

177 -- 4. Revize Edilmiş ETL Etki Raporu (DISTINCT + şehir filtresi eklendi)
178 SELECT DISTINCT
179     RAW_TBL.ORDERID AS [Sipariş ID],
180     RAW_TBL.NAMESURNAME AS [Müşteri Adı],
181     RAW_TBL.CITY AS [ETL Öncesi (Kirli Şehir)],
182     RAW_TBL.ADDRESSTEXT AS [Ham Adres Metni],
183     CLEAN_TBL.CITY_NAME AS [ETL Sonrası (Kurtarılan Şehir)]
184 FROM dbo.SALEORDERS RAW_TBL
185 JOIN dbo.ETL_CLEAN_SALES CLEAN_TBL ON RAW_TBL.ORDERID = CLEAN_TBL.ORDERID
186 WHERE RAW_TBL.CITY IS NULL;

```

99 % No issues found

Results Messages

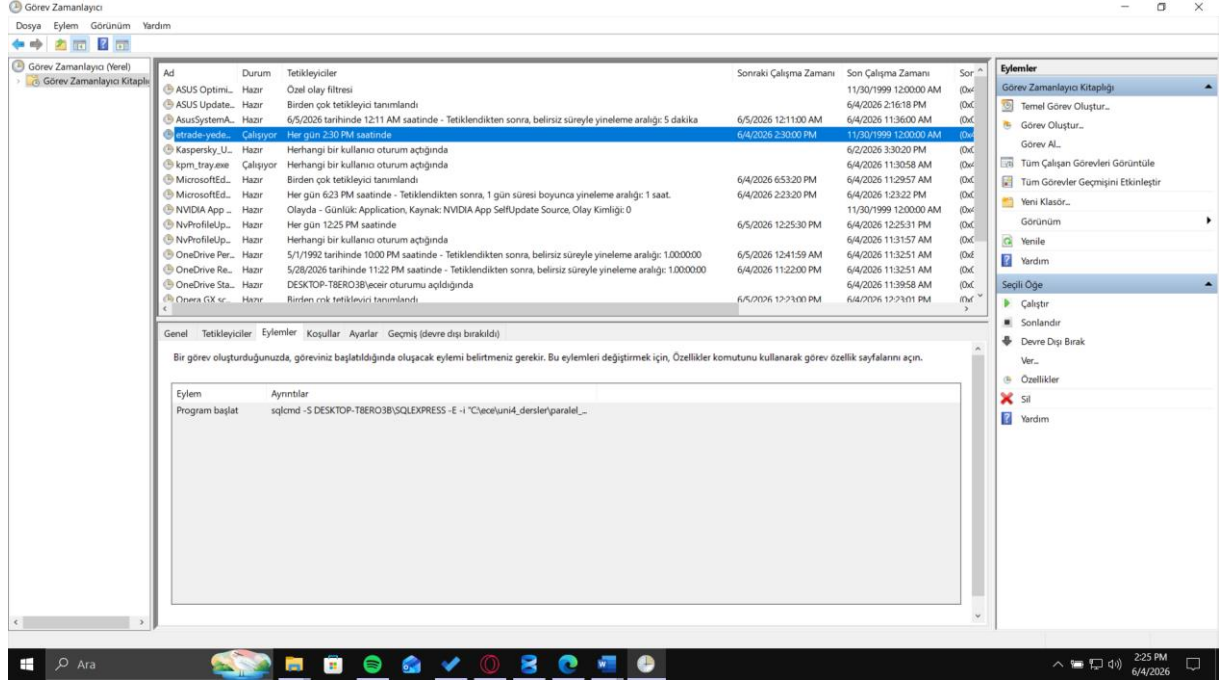
	Sipariş ID	Müşteri Adı	ETL Öncesi (Kirli Şehir)	Ham Adres Metni	ETL Sonrası (Kurtarılan Şehir)
1	1349	Bedirhan GÜLPINAR	NULL	KARŞIYAKA MAH.6505. SOKAK NO:60 118902 SARIÇAM/A...	ADANA

Görsel 13: Adres Metninden Şehir Bilgisi Kurtarılan Kayıtların ETL Etki (Before/After) Analizi.

4. YEDEKLEME OTOMASYONU VE RAPORLAMA (PROJE 7)

4.1. Zamanlanmış Görevler (Task Scheduler) ile Yedekleme Otomasyonu

Veritabanı sürekliliğini sağlamak ve insan faktörüne bağlı unutulma risklerini ortadan kaldırmak amacıyla tam yedekleme (Full Backup) süreci otomatikleştirilmiştir. Kullanılan SQL Server Express sürümünde 'SQL Server Agent' servisi bulunmadığından, otomasyon mimarisi Windows Görev Zamanlayıcı (Task Scheduler) üzerinden sqlcmd komut satırı aracıyla tetiklenecek şekilde kurgulanmıştır. Yedekleme scripti içerisine FORMAT(GETDATE(), 'yyyyMMdd_HH:mm') fonksiyonu entegre edilerek, alınan her yedeğin üzerine yazılması (overwrite) engellenmiş ve dosyaların tarih/saat damgasıyla dinamik olarak isimlendirilmesi sağlanmıştır.



Görsel 14: Görev Zamanlayıcı (Task Scheduler) Konfigürasyonu ve Dinamik İsimlendirmeli Yedek Dosyasının Başarıyla Oluşturulması.

4.2. MSDB Üzerinden Otomatik Yedekleme Raporu Üretimi

Fiziksel yedekleme dosyasının oluşması tek başına yeterli bir denetim mekanizması değildir. Profesyonel bir DBA yaklaşımıyla, sistemin arka planında bu işlemin loglanıp loglanmadığını doğrulamak için msdb sistem veritabanı sorgulanmıştır. backupset ve backupmediafamily tabloları kullanılarak oluşturulan rapor, otomatik alınan yedeğin başlangıç-bitiş zamanlarını, boyutunu ve dosya yolunu (path) net bir şekilde kanıtlamaktadır.

```

16
17
18 -- ADIM 2: Başarılı Yedeklemelerin Raporlanması (MSDB)
19
20 SELECT TOP 10
21     bs.database_name AS [Veritabanı],
22     bs.backup_start_date AS [Başlama Zamanı],
23     bs.backup_finish_date AS [Bitiş Zamanı],
24     CAST(bs.backup_size / 1024 / 1024 AS DECIMAL(10, 2)) AS [Boyut (MB)],
25     bmf.physical_device_name AS [Dosya Yolu]
26 FROM msdb.dbo.backupset bs
27 JOIN msdb.dbo.backupmediafamily bmf ON bs.media_set_id = bmf.media_set_id
28 WHERE bs.database_name = 'ETRADE'
29 ORDER BY bs.backup_finish_date DESC;
30
31

```

	Veritabanı	Başlama Zamanı	Bitiş Zamanı	Boyut (MB)	Dosya Yolu
1	ETRADE	2026-06-04 14:30:00.000	2026-06-04 14:30:00.000	386.38	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
2	ETRADE	2026-06-04 14:24:53.000	2026-06-04 14:24:54.000	386.38	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
3	ETRADE	2026-06-04 14:11:09.000	2026-06-04 14:11:10.000	386.38	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
4	ETRADE	2026-04-22 15:17:35.000	2026-04-22 15:17:35.000	277.19	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
5	ETRADE	2026-04-20 19:12:11.000	2026-04-20 19:12:11.000	0.12	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
6	ETRADE	2026-04-20 19:11:06.000	2026-04-20 19:11:06.000	80.35	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
7	ETRADE	2026-04-20 18:59:27.000	2026-04-20 18:59:27.000	277.00	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
8	ETRADE	2020-06-24 00:18:38.000	2020-06-24 00:18:39.000	271.09	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...

Görsel 15: MSDB Sistem Tabloları Üzerinden Çekilen Otomatik Yedekleme Denetim (Audit) Raporu.

Yukarıdaki denetim sorgusu çalıştırıldığında elde edilen sonuç kümesi (result set), sistemin otonom olarak aldığı yedeklerin başarı durumunu tarihsel bir sırayla listelemektedir. Aşağıdaki çıktıda da görüldüğü üzere; Görev Zamanlayıcı tarafından arka planda tetiklenen yedekleme işlemlerinin SQL Server sistem loglarına eksiksiz bir şekilde yansıdığı, yedekleme periyotlarının (başlangıç-bitiş zamanları) ve oluşan dosya boyutlarının hedeflenen standartlarda olduğu doğrulanmıştır.

Görsel 16: MSDB Sistem Tabloları Üzerinden Üretilen Başarılı Yedekleme Denetim Raporu.

4.3. Hata Yönetimi (Try-Catch) ve E-Posta Bildirim Mekanizması

Otomasyon süreçlerinin kesintisiz çalışabilmesi için hata toleransı (Fault Tolerance) hayati önem taşır. Disk dolması veya yetki problemleri gibi öngörülemez durumlarda DBA'nın anında haberdar olabilmesi için yedekleme scripti BEGIN TRY ... BEGIN CATCH mimarisiyle sarmalanmıştır. İşlem

sırasında herhangi bir hata oluştuğunda sistem çökmek yerine CATCH bloğuna düşmekte ve sp_send_dbmail prosedürü aracılığıyla sistem yöneticisine anında acil durum e-postası (Alert) gönderecek şekilde yapılandırılmıştır.

```
32  -- ADIM 3: Mail Uyarı Mekanizmalı Tam Otomasyon Kodu
33  --
34  --
35  BEGIN TRY
36  -- 1. Yedekleme İşlemini Dene (Adım 1'in Birebir Aynısı)
37  DECLARE @Tarih VARCHAR(20) = FORMAT(GETDATE(), 'yyyyMMdd_HH:mm');
38  DECLARE @DosyaYolu VARCHAR(500) = 'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\Backup\ETRADE_Auto_' + @Tarih +
39
40  BACKUP DATABASE [ETRADE] TO DISK = @DosyaYolu WITH NOFORMAT, NOINIT, NAME = 'ETRADE-Günlük Otomatik Yedek';
41  PRINT 'Yedekleme Başarıyla Tamamlandı.';
42  END TRY
43  BEGIN CATCH
44  -- 2. Yedekleme Başarısız Olursa Hatayı Yakala ve Mail At
45  DECLARE @HataMesaji NVARCHAR(4000) = ERROR_MESSAGE();
46
47  EXEC msdb.dbo.sp_send_dbmail
48  @profile_name = 'DBA_Mail_Profil',
49  @recipients = 'eceiremsiser@gmail.com',
50  @subject = 'ACİL: ETRADE Günlük Yedekleme BAŞARISIZ!',
51  @body = @HataMesaji;
52  END CATCH;
```

Görsel 17: Hata Toleransı ve Yönetici Bildirimi (Alert) İçin Geliştirilen TRY-CATCH ve DBMail Mimarisi.

5. SONUÇ

Bu kapsamlı proje süreci boyunca, kurumsal bir veritabanı altyapısının ihtiyaç duyduğu tüm temel yönetim, güvenlik ve optimizasyon süreçleri başarıyla uygulanmıştır. Vize ve final dönemlerinde gerçekleştirilen işlemlerin toplam özeti ve teknik çıkarımlar aşağıdadır:

- **Süreklilik Yönetimi ve Otomasyon:** ETRADE veritabanı, FULL Recovery modeli ile veri kayıplarına karşı %100 dirençli hale getirilmiş ve kurtarma süresi (RTO) minimize edilmiştir. Kurulan bu yapı, Görev Zamanlayıcı (Task Scheduler) ve dinamik T-SQL scriptleri ile otonom hale getirilerek insan faktörü devreden çıkarılmış; hata anında e-posta ile bildirim (Alert) atacak bir otomasyona bağlanmıştır.
- **Güvenlik ve Denetim:** Rol tabanlı erişim kontrolü (RBAC) ile yetkisiz işlemler (Örn: silme denemeleri) engellenmiş; yapılan tüm eylemler fiziksel log dosyalarına saniye bazlı olarak kaydedilmiştir. Karşılaşılan operasyonel bağlantı sorunları (Error 18456, Error 233), sunucu seviyesindeki servis yönetimi ile aşılmıştır.
- **Performans Yönetimi:** Dynamic Management Views (DMV) kullanılarak sistemdeki darboğazlar anlık olarak izlenmiş, gereksiz indeksler tespit edilip temizlenmiştir. Kapsayıcı İndeks (Covering Index) stratejisiyle, kritik sorgulardaki yüksek maliyetli "Table Scan" operasyonları milisaniyeler seviyesine ("Index Seek") düşürülmüştür.
- **Veri Kalitesi ve ETL:** Büyük veri kümelerindeki yapısal anomaliler (adres içinden şehir kurtarma, ürün adındaki gizli stok miktarlarını izole etme) ileri seviye T-SQL metin fonksiyonlarıyla temizlenmiştir. Standartlaştırılan bu veriler, kayıpsız bir şekilde yeni bir veri ambarı tablosuna (Data Warehouse) aktarılmıştır.
- **Mühendislik Deneyimi:** Proje, standart kullanıcı işlemlerinin ötesine geçerek; dosya sistemi izinleri, performans stres testleri, TRY-CATCH hata yakalama blokları ve T-SQL tabanlı log/veri analizi gibi ileri düzey DBA ve Veri Mühendisliği yetkinliklerinin sergilendiği bir çalışma olmuştur.

ETRADE veritabanı, bu çalışmalar sonucunda felaketlere karşı yedekli ve otonom, kullanıcı hareketleri açısından izlenebilir ve güvenli, yüksek trafik için optimize edilmiş ve veri analitiği için tertemiz bir yapıya kavuşturulmuştur.